JSC INTERNATIONAL INFORMATION TECHNOLOGY UNIVERSITY

Department of Information Systems

LABORATORY WORK No. 7

REST API DESIGN

DTO records - HTTP semantics - Problem Details - OpenAPI

Course: RWPSF 3305
Web Application Development with Spring Framework

Performed by: Kassymzhanov Arzum
Group: IT1-2404IS
Variant: 12 - RestTestClient

Almaty, 2026

Contents

1. Aim and project baseline
2. REST contract
3. DTO layer and mapper
4. REST controller
5. HTTP verification
6. Centralized error handling
7. OpenAPI and Swagger UI
8. Individual assignment - Variant 12
9. Automated verification
10. Defence questions
11. Conclusions
12. References

1. Aim and project baseline

Aim. To design and implement a REST API for the Book resource using explicit DTOs, correct HTTP semantics, Bean Validation, RFC 9457 Problem Details and OpenAPI documentation.

Repository: https://github.com/Karz317/spring-lab-01/tree/lab07. The branch lab07 continues the completed laboratory 6 code and preserves the MVC catalogue, localization and previous tests.

Item	Actual value
Student	Kassymzhanov Arzum
Group	IT1-2404IS
Variant	12 - RestTestClient
Java / Spring Boot	JDK 25.0.2 / Spring Boot 4.1.1
OpenAPI	springdoc-openapi-starter-webmvc-ui 3.1.1
Storage	In-memory ConcurrentHashMap with three initial books
Verification	36 automated tests, 11 live HTTP observations, 3 browser screenshots

The implementation exposes only BookRequest and BookResponse at the HTTP boundary. The domain record retains internal state such as availability, while the API publishes id and createdAt as read-only response data.

2. REST contract

The API uses resource-oriented paths and status codes that describe the outcome of each operation.

Method and path	Request body	Success	Error responses
GET /api/books	-	200 List<BookResponse>	-
GET /api/books/{id}	-	200 BookResponse	404 ProblemDetail
POST /api/books	BookRequest	201 BookResponse + Location	400 validation; 409 duplicate
PUT /api/books/{id}	BookRequest	200 BookResponse	400; 404; 409
DELETE /api/books/{id}	-	204, empty body	404 ProblemDetail
GET /api/books/search?q=...	-	200 List<BookResponse>	400 method validation

- POST returns 201 Created and a Location header containing the URI of the created book.
- DELETE returns 204 No Content with no response body.
- A missing identifier produces 404; a duplicate title-author pair produces 409 Conflict.
- Validation failures and application errors use application/problem+json.

3. DTO layer and mapper

BookRequest is the write contract. The client supplies title, author, year and genre, but cannot assign an identifier, creation timestamp or internal availability flag. BookResponse is the read contract and contains id and createdAt. This separation prevents mass assignment and allows the domain model or storage representation to change without silently breaking clients.

Field	BookRequest rule	BookResponse
title	@NotBlank; @Size(max=120)	Returned
author	@NotBlank; @NoDigits	Returned
year	@Min(1450); @Max(2100)	Returned
genre	@NotNull	Returned
id	Not accepted	Returned
createdAt	Not accepted	Returned

Listing 1. BookRequest record

src/main/java/kz/iitu/springlab/web/dto/BookRequest.java

```
package kz.iitu.springlab.web.dto;

import jakarta.validation.constraints.Max;
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Size;
import kz.iitu.springlab.web.Genre;
import kz.iitu.springlab.web.validation.NoDigits;

public record BookRequest(
        @NotBlank(message = "{book.title.required}")
        @Size(max = 120, message = "{book.title.size}")
        String title,

        @NotBlank(message = "{book.author.required}")
        @NoDigits
        String author,

        @Min(value = 1450, message = "{book.year.range}")
        @Max(value = 2100, message = "{book.year.range}")
        int year,

        @NotNull(message = "{book.genre.required}")
        Genre genre) {
}
```

Listing 2. BookResponse record

src/main/java/kz/iitu/springlab/web/dto/BookResponse.java

```
package kz.iitu.springlab.web.dto;

import java.time.Instant;
import kz.iitu.springlab.web.Genre;

public record BookResponse(
        Long id,
        String title,
        String author,
        int year,
        Genre genre,
        Instant createdAt) {
}
```

Listing 3. BookMapper

src/main/java/kz/iitu/springlab/web/dto/BookMapper.java

```
package kz.iitu.springlab.web.dto;
```

```
import java.time.Instant;
import java.util.List;
import kz.iitu.springlab.catalog.Book;
import org.springframework.stereotype.Component;

@Component
public class BookMapper {
    public Book toDomain(BookRequest request) {
        return new Book(null, request.title(), request.author(), request.year(), request.genre(),
                true, Instant.now());
    }

    public BookResponse toResponse(Book book) {
        return new BookResponse(book.id(), book.title(), book.author(), book.year(), book.genre(),
                book.createdAt());
    }

    public List<BookResponse> toResponseList(List<Book> books) {
        return books.stream().map(this::toResponse).toList();
    }
}
```

4. REST controller

BookApiController is mapped to /api/books. It delegates business operations to BookService and uses BookMapper in both directions. The controller never exposes Book directly. OpenAPI annotations describe the resource and expected responses; validation remains executable code on the DTO.

Listing 4. BookApiController

src/main/java/kz/iitu/springlab/web/BookApiController.java

```
package kz.iitu.springlab.web;

import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.Parameter;
import io.swagger.v3.oas.annotations.media.Content;
import io.swagger.v3.oas.annotations.media.Schema;
import io.swagger.v3.oas.annotations.responses.ApiResponse;
import io.swagger.v3.oas.annotations.responses.ApiResponses;
import io.swagger.v3.oas.annotations.tags.Tag;
import jakarta.validation.Valid;
import jakarta.validation.constraints.Size;
import java.net.URI;
import java.util.List;
import kz.iitu.springlab.catalog.Book;
import kz.iitu.springlab.catalog.BookService;
import kz.iitu.springlab.web.dto.BookMapper;
import kz.iitu.springlab.web.dto.BookRequest;
import kz.iitu.springlab.web.dto.BookResponse;
import org.springframework.http.ProblemDetail;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

@Tag(name = "Books", description = "The catalogue of books")
@RestController
@RequestMapping("/api/books")
public class BookApiController {
    private final BookService service;
    private final BookMapper mapper;

    public BookApiController(BookService service, BookMapper mapper) {
        this.service = service;
        this.mapper = mapper;
    }

    @Operation(summary = "List books")
    @ApiResponse(responseCode = "200", description = "The collection was returned")
    @GetMapping
    public List<BookResponse> list(@RequestParam(required = false) String author) {
        return mapper.toResponseList(service.findAll(author));
    }

    @Operation(summary = "Search books by title")
    @ApiResponses({
            @ApiResponse(responseCode = "200", description = "The matching books were returned"),
            @ApiResponse(responseCode = "400", description = "The query length is invalid",
                    content = @Content(schema = @Schema(implementation = ProblemDetail.class)))
    })
    @GetMapping("/search")
    public List<BookResponse> search(
            @RequestParam(defaultValue = "")
            @Size(min = 2, max = 60, message = "{book.search.size}") String q) {
        return mapper.toResponseList(service.search(q));
    }

    @Operation(summary = "Find a book by its identifier")
    @ApiResponses({
            @ApiResponse(responseCode = "200", description = "The book was found"),
            @ApiResponse(responseCode = "404", description = "No book with this id",
                    content = @Content(schema = @Schema(implementation = ProblemDetail.class)))
    })
    @GetMapping("/{id}")
    public BookResponse find(
            @Parameter(description = "Identifier of the book", example = "42")
            @PathVariable long id) {
        return mapper.toResponse(service.findById(id));
    }

    @Operation(summary = "Create a book")
```

```
    @ApiResponses({
            @ApiResponse(responseCode = "201", description = "The book was created"),
            @ApiResponse(responseCode = "400", description = "The request body is invalid",
                    content = @Content(schema = @Schema(implementation = ProblemDetail.class))),
            @ApiResponse(responseCode = "409", description = "The title and author already exist",
                    content = @Content(schema = @Schema(implementation = ProblemDetail.class)))
    })
    @PostMapping
    public ResponseEntity<BookResponse> create(@Valid @RequestBody BookRequest request) {
        Book saved = service.create(mapper.toDomain(request));
        return ResponseEntity.created(URI.create("/api/books/" + saved.id()))
                .body(mapper.toResponse(saved));
    }

    @Operation(summary = "Replace a book")
    @ApiResponses({
            @ApiResponse(responseCode = "200", description = "The book was updated"),
            @ApiResponse(responseCode = "400", description = "The request body is invalid",
                    content = @Content(schema = @Schema(implementation = ProblemDetail.class))),
            @ApiResponse(responseCode = "404", description = "No book with this id",
                    content = @Content(schema = @Schema(implementation = ProblemDetail.class))),
            @ApiResponse(responseCode = "409", description = "The title and author already exist",
                    content = @Content(schema = @Schema(implementation = ProblemDetail.class)))
    })
    @PutMapping("/{id}")
    public BookResponse update(
            @Parameter(description = "Identifier of the book", example = "42")
            @PathVariable long id,
            @Valid @RequestBody BookRequest request) {
        return mapper.toResponse(service.update(id, mapper.toDomain(request)));
    }

    @Operation(summary = "Delete a book")
    @ApiResponses({
            @ApiResponse(responseCode = "204", description = "The book was deleted"),
            @ApiResponse(responseCode = "404", description = "No book with this id",
                    content = @Content(schema = @Schema(implementation = ProblemDetail.class)))
    })
    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(
            @Parameter(description = "Identifier of the book", example = "42")
            @PathVariable long id) {
        service.delete(id);
        return ResponseEntity.noContent().build();
    }
}
```

5. HTTP verification

The executable JAR was started on port 8080 and the following requests were made against the live application. Values shown below are captured in docs/lab07-observations.json.

Request	Content-Type	Observed result
GET /api/books	application/json	200
GET /api/books/1	application/json	200; Effective Java
POST /api/books	application/json	201; Location: /api/books/4; HTTP Evidence 1792426322402078400
GET /api/books/4	application/json	200; HTTP Evidence 1792426322402078400
DELETE /api/books/4	-	204; empty body
DELETE /api/books/4	application/problem+json	404; Book not found

Created resource body

```
{
  "id": 4,
  "title": "HTTP Evidence 1792426322402078400",
  "author": "Evidence Author",
  "year": 2026,
  "genre": "SCIENCE",
  "createdAt": "2026-10-19T16:12:02.468077400Z"
}
```

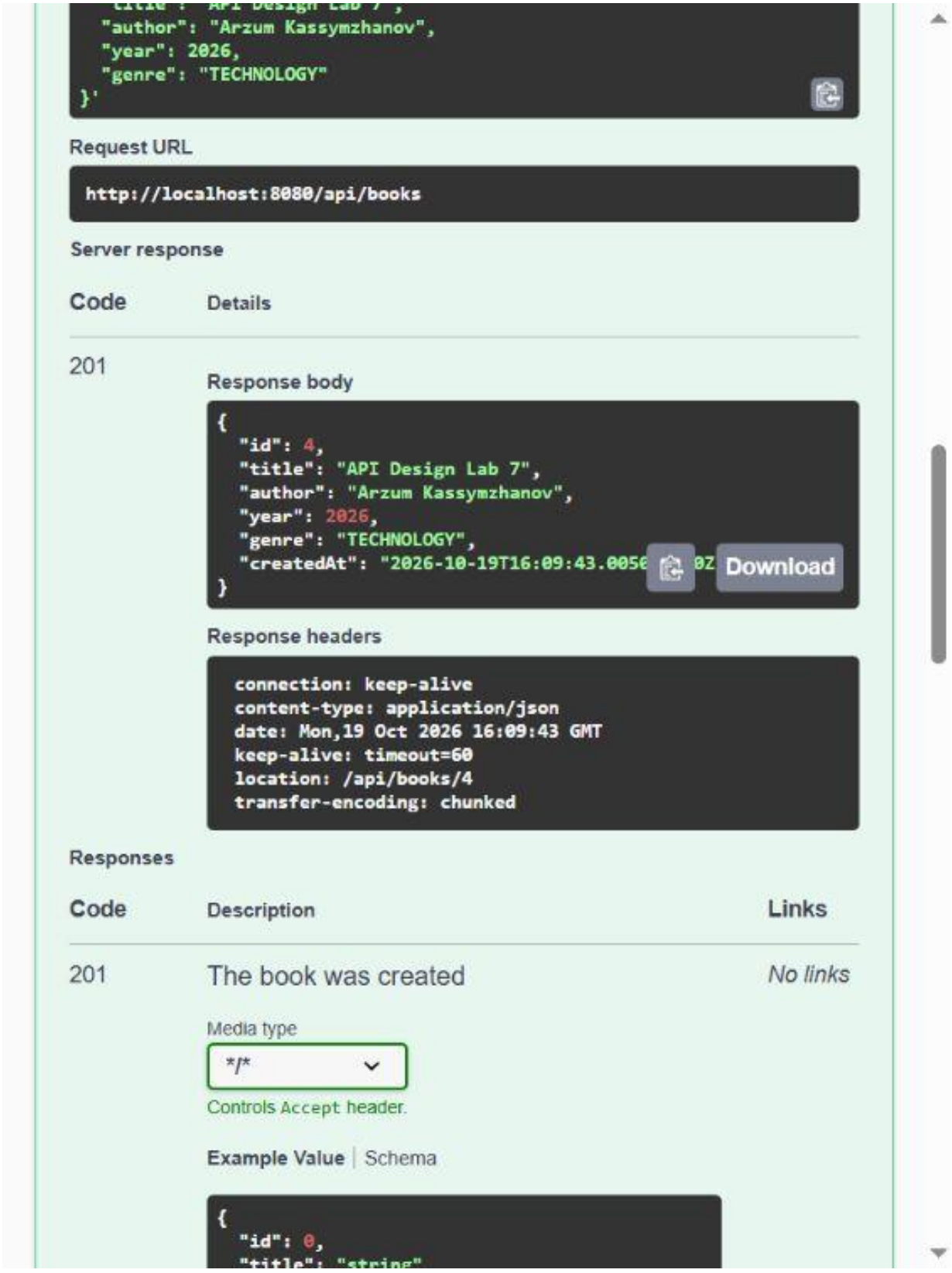


Figure 1. Live POST /api/books response: 201 Created, response DTO and Location header.

6. Centralized error handling

ApiExceptionHandler extends ResponseEntityExceptionHandler and is restricted to REST controllers. Known application exceptions become ProblemDetail responses; invalid request bodies are represented by a structured errors array naming each rejected property.

Scenario	Status / media type	Handler result
Unknown id	404 application/problem+json	BookNotFoundException -> ProblemDetail
Invalid BookRequest	400 application/problem+json	errors[].field and errors[].message
Duplicate title and author	409 application/problem+json	DuplicateTitleException -> ProblemDetail

Complete 404 response body:

```
{
  "detail": "Book with id 999 was not found",
  "instance": "/api/books/999",
  "status": 404,
  "title": "Book not found",
  "type": "https://api.iitu.kz/errors/book-not-found"
}
```

Validation response (the rejected property is visible as errors[0].field):

```
{
  "detail": "One or more request fields are invalid",
  "instance": "/api/books",
  "status": 400,
  "title": "Validation failed",
  "errors": [
    {
      "field": "title",
      "message": "The title must not be empty"
    }
  ]
}
```

Duplicate response:

```
{
  "detail": "A book with title 'Effective Java' and the same author already exists",
  "instance": "/api/books",
  "status": 409,
  "title": "Duplicate title",
  "type": "https://api.iitu.kz/errors/duplicate-title",
  "conflictingTitle": "Effective Java"
}
```

Listing 5. BookNotFoundException

src/main/java/kz/iitu/springlab/catalog/BookNotFoundException.java

```
package kz.iitu.springlab.catalog;

public class BookNotFoundException extends RuntimeException {
    private final long id;

    public BookNotFoundException(long id) {
        super("Book with id " + id + " was not found");
        this.id = id;
    }

    public long getId() {
        return id;
    }
}
```

Listing 6. DuplicateTitleException

src/main/java/kz/iitu/springlab/service/DuplicateTitleException.java

```
package kz.iitu.springlab.service;

public class DuplicateTitleException extends RuntimeException {
```

```
    private final String title;

    public DuplicateTitleException(String title) {
        super("A book with title '" + title + "' and the same author already exists");
        this.title = title;
    }

    public String getTitle() {
        return title;
    }
}
```

Listing 7. ApiExceptionHandler

src/main/java/kz/iitu/springlab/web/ApiExceptionHandler.java

```
package kz.iitu.springlab.web;

import java.net.URI;
import java.util.Locale;
import java.util.Map;
import kz.iitu.springlab.catalog.BookNotFoundException;
import kz.iitu.springlab.service.DuplicateTitleException;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.context.MessageSource;
import org.springframework.context.i18n.LocaleContextHolder;
import org.springframework.http.HttpHeaders;
import org.springframework.http.HttpStatus;
import org.springframework.http.HttpStatusCode;
import org.springframework.http.ProblemDetail;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.bind.annotation.RestControllerAdvice;
import org.springframework.web.context.request.WebRequest;
import org.springframework.web.method.annotation.HandlerMethodValidationException;
import org.springframework.web.servlet.mvc.method.annotation.ResponseEntityExceptionHandler;

@RestControllerAdvice(annotations = RestController.class)
public class ApiExceptionHandler extends ResponseEntityExceptionHandler {
    private static final Logger log = LoggerFactory.getLogger(ApiExceptionHandler.class);
    private final MessageSource messages;

    public ApiExceptionHandler(MessageSource messages) {
        this.messages = messages;
    }

    @ExceptionHandler(BookNotFoundException.class)
    public ProblemDetail handleNotFound(BookNotFoundException exception) {
        ProblemDetail problem = ProblemDetail.forStatusAndDetail(
                HttpStatus.NOT_FOUND, exception.getMessage());
        problem.setType(URI.create("https://api.iitu.kz/errors/book-not-found"));
        problem.setTitle("Book not found");
        return problem;
    }

    @ExceptionHandler(DuplicateTitleException.class)
    public ProblemDetail handleDuplicate(DuplicateTitleException exception) {
        ProblemDetail problem = ProblemDetail.forStatusAndDetail(
                HttpStatus.CONFLICT, exception.getMessage());
        problem.setType(URI.create("https://api.iitu.kz/errors/duplicate-title"));
        problem.setTitle("Duplicate title");
        problem.setProperty("conflictingTitle", exception.getTitle());
        return problem;
    }

    @Override
    protected ResponseEntity<Object> handleMethodArgumentNotValid(
            MethodArgumentNotValidException exception,
            HttpHeaders headers,
            HttpStatusCode status,
            WebRequest request) {
        ProblemDetail problem = ProblemDetail.forStatusAndDetail(
                status, "One or more request fields are invalid");
        problem.setTitle("Validation failed");
        problem.setProperty("errors", exception.getBindingResult().getFieldErrors().stream()
                .map(error -> Map.of(
                        "field", error.getField(),
                        "message", error.getDefaultMessage() == null
                                ? "Invalid value" : error.getDefaultMessage()))
                .toList());
        return handleExceptionInternal(exception, problem, headers, status, request);
    }

    @Override
    protected ResponseEntity<Object> handleHandlerMethodValidationException(
```

```
            HandlerMethodValidationException exception,
            HttpHeaders headers,
            HttpStatusCode status,
            WebRequest request) {
        Locale locale = LocaleContextHolder.getLocale();
        log.info("Handled {} in book search", exception.getClass().getSimpleName());
        ProblemDetail problem = ProblemDetail.forStatusAndDetail(
                status, messages.getMessage("book.search.size", null, locale));
        problem.setTitle(messages.getMessage("error.validation", null, locale));
        problem.setProperty("exception", exception.getClass().getSimpleName());
        problem.setProperty("errors", exception.getParameterValidationResults().stream()
                .flatMap(result -> result.getResolvableErrors().stream().map(error -> Map.of(
                        "parameter", result.getMethodParameter().getParameterName(),
                        "message", messages.getMessage(error, locale))))
                .toList());
        return handleExceptionInternal(exception, problem, headers, status, request);
    }
}
```

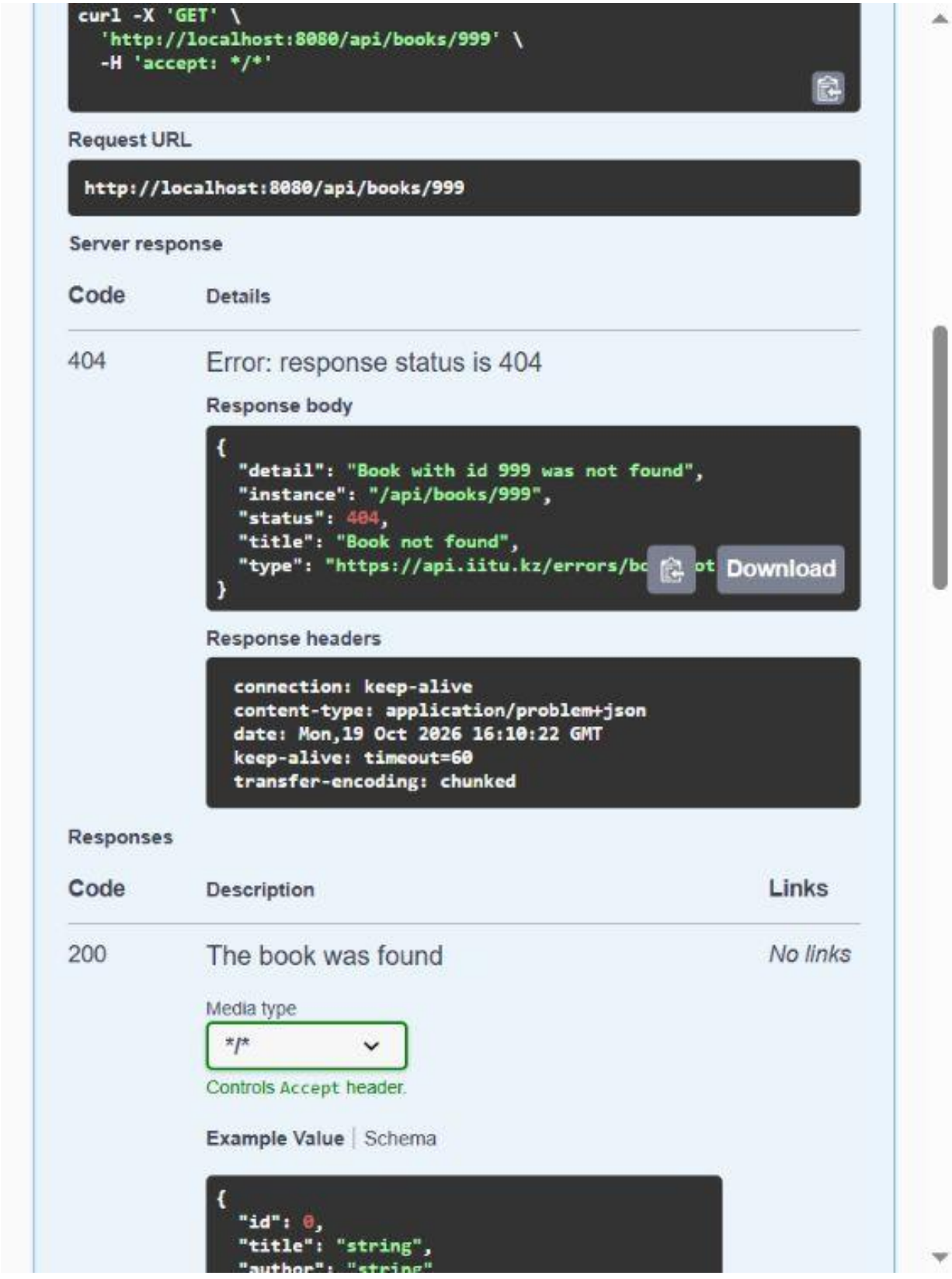


Figure 2. Swagger UI executes GET /api/books/999 and displays the live 404 ProblemDetail.

7. OpenAPI and Swagger UI

The springdoc starter generates an OpenAPI 3.1 document at /v3/api-docs and an interactive page at /swagger-ui.html. The generated document contains /api/books, /api/books/{id} and /api/books/search. Bean Validation metadata is reused in the schema: title has maxLength 120, year has minimum 1450 and maximum 2100, and genre is an enum.

Path	Generated operations
/api/books	GET, POST
/api/books/{id}	GET, PUT, DELETE
/api/books/search	GET

OpenAPI dependency

```
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
  <version>3.1.1</version>
</dependency>
```

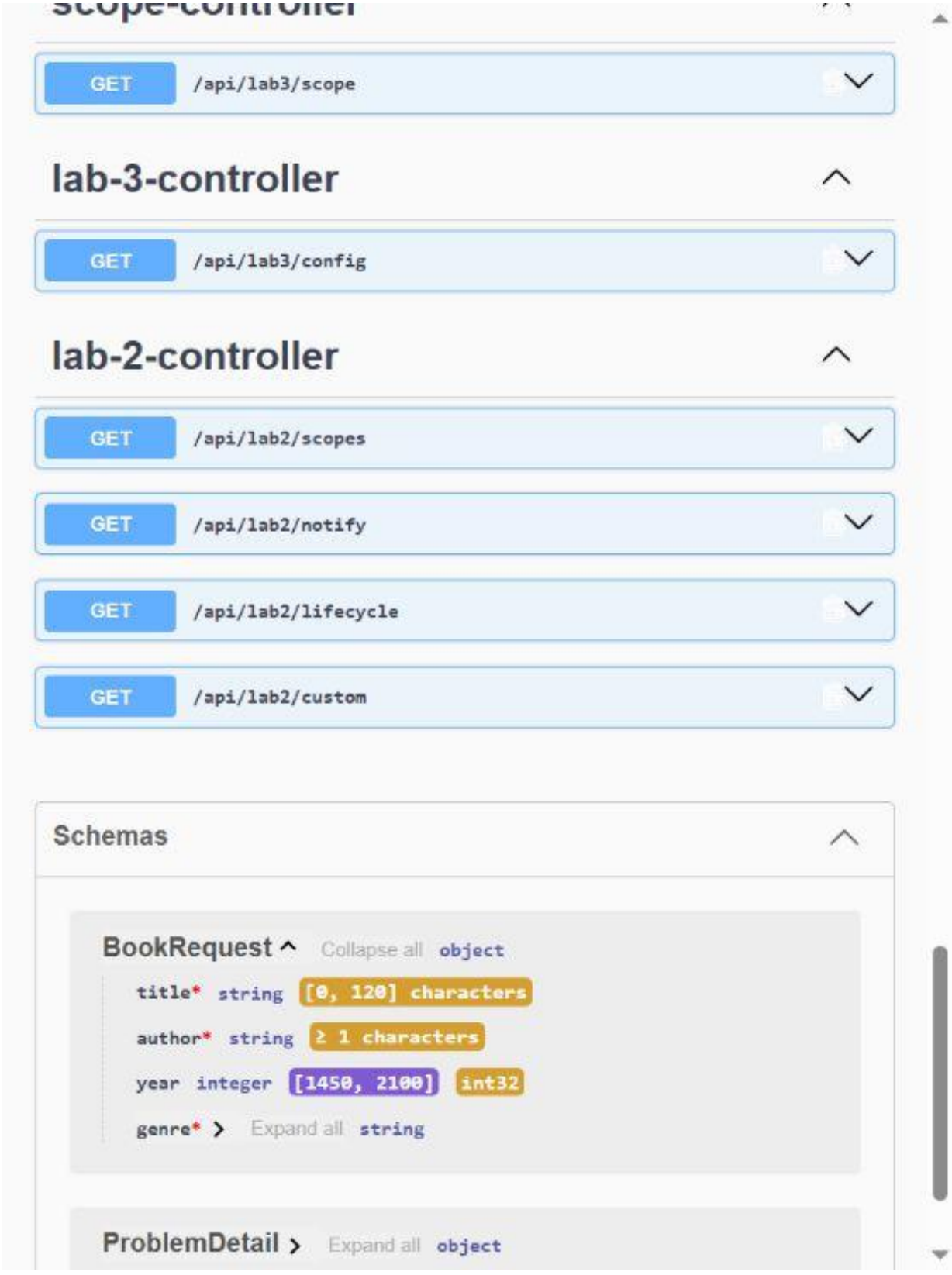


Figure 3. Generated BookRequest schema in Swagger UI, including validation limits.

8. Individual assignment - Variant 12

Variant 12 requires three tests with Spring RestTestClient. The client is bound to the real WebApplicationContext, so Spring MVC request mapping, validation, exception advice, message conversion and response headers are exercised without starting an external server.

Test	Required assertion	Result
createReturns201AndLocation	201, Location exists, created DTO can be read	Passed
missingBookReturns404ProblemDetail	404, application/problem+json, standard fields	Passed
deleteReturns204AndNoBody	204 and empty response body	Passed

Listing 8. RestTestClient tests

src/test/java/kz/iitu/springlab/BookApiRestTestClientTests.java

```
package kz.iitu.springlab;

import java.net.URI;
import java.util.UUID;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.http.MediaType;
import org.springframework.test.web.servlet.client.RestTestClient;
import org.springframework.web.context.WebApplicationContext;

import static org.assertj.core.api.Assertions.assertThat;

@SpringBootTest
class BookApiRestTestClientTests {
    private RestTestClient client;

    @BeforeEach
    void setUp(WebApplicationContext context) {
        client = RestTestClient.bindToApplicationContext(context).build();
    }

    @Test
    void createReturns201AndLocation() {
        String title = "RestTestClient " + UUID.randomUUID();

        var result = client.post().uri("/api/books")
                .contentType(MediaType.APPLICATION_JSON)
                .body(json(title))
                .exchange()
                .expectStatus().isCreated()
                .expectHeader().exists("Location")
                .expectBody()
                .jsonPath("$.title").isEqualTo(title)
                .jsonPath("$.createdAt").exists()
                .returnResult();

        URI location = result.getResponseHeaders().getLocation();
        assertThat(location).isNotNull();
        client.get().uri(location.toString())
                .exchange()
                .expectStatus().isOk()
                .expectBody()
                .jsonPath("$.title").isEqualTo(title);
    }

    @Test
    void missingBookReturns404ProblemDetail() {
        client.get().uri("/api/books/999999")
                .accept(MediaType.APPLICATION_PROBLEM_JSON)
                .exchange()
                .expectStatus().isNotFound()
                .expectHeader().contentTypeCompatibleWith(MediaType.APPLICATION_PROBLEM_JSON)
                .expectBody()
                .jsonPath("$.status").isEqualTo(404)
                .jsonPath("$.title").isEqualTo("Book not found")
                .jsonPath("$.instance").isEqualTo("/api/books/999999");
    }

    @Test
    void deleteReturns204AndNoBody() {
        var created = client.post().uri("/api/books")
                .contentType(MediaType.APPLICATION_JSON)
                .body(json("Delete " + UUID.randomUUID()))
```

```
                .exchange()
                .expectStatus().isCreated()
                .expectBody()
                .returnResult();
        URI location = created.getResponseHeaders().getLocation();
        assertThat(location).isNotNull();

        client.delete().uri(location.toString())
                .exchange()
                .expectStatus().isNoContent()
                .expectBody().isEmpty();
    }

    private String json(String title) {
        return """
                {
                  "title": "%s",
                  "author": "Test Author",
                  "year": 2026,
                  "genre": "TECHNOLOGY"
                }
                """.formatted(title);
    }
}
```

9. Automated verification

The complete project was verified with .\mvnw.cmd -q clean verify. Maven finished successfully and produced the executable JAR. Test reports contain no failures, errors or skipped tests.

Test suite	Tests	Failures	Errors	Skipped
BookApiRestTestClientTests	3	0	0	0
BookWorkflowTests	26	0	0	0
NoDigitsValidatorTests	6	0	0	0
SpringLab01ApplicationTests	1	0	0	0
Total	36	0	0	0

Reproduction commands (PowerShell)

```
.\mvnw.cmd clean verify
java -jar .\target\spring-lab-01-0.0.1-SNAPSHOT.jar
```

The live verification additionally confirmed 200 list and item responses, 201 creation and follow-up GET, 204 deletion, 404 after repeated deletion, 400 body validation, 409 duplicate detection, 400 search-parameter validation and the generated OpenAPI schema.

10. Defence questions

Why not accept Book directly? It would expose server-owned fields such as id, createdAt and internal state to mass assignment and couple clients to the domain model.

What changes if storage changes? The DTO contract can remain stable while the domain object, database entity or mapper changes internally.

Why does POST return 201 and Location? 201 states that a resource was created; Location tells the client where the new resource can be retrieved.

Why does DELETE return 204? A successful delete has no representation to return, so an empty body is explicit and efficient.

What is ProblemDetail? It is Spring's representation of the RFC 9457 problem-details format with type, title, status, detail and instance fields.

Why use 409 for duplicates? The request is syntactically valid but conflicts with the current catalogue state.

Where does maxLength 120 in OpenAPI come from? springdoc reads the @Size(max=120) Bean Validation annotation on BookRequest.title.

Why use ResponseEntity for create and delete? Those operations must control the status and headers explicitly: Location for 201 and an empty 204 response.

Does Swagger UI replace tests? No. It is an interactive client and contract explorer; automated tests remain repeatable regression checks.

What does RestTestClient verify here? It exercises the full MVC layer through the application context, including serialization, validation, advice and headers.

11. Conclusions

The implemented API provides complete CRUD operations for books with stable request and response contracts. DTOs add mapping code, but they prevent clients from controlling server-owned fields and keep persistence changes away from the public API. Correct status codes, Location headers and ProblemDetail responses make success and failure semantics unambiguous. OpenAPI generation and the RestTestClient suite give both interactive documentation and repeatable evidence that the contract is implemented.

12. References

1. Laboratory_work_7_EN.docx - primary assignment and report requirements.
2. Presentation_Lecture_7_EN.pptx - lecture material on REST semantics, DTOs, errors and OpenAPI.
3. Project source code: https://github.com/Karz317/spring-lab-01/tree/lab07
4. OpenAPI document generated by the application: http://localhost:8080/v3/api-docs
5. Swagger UI generated by the application: http://localhost:8080/swagger-ui.html